

Notification Template Engine

Project Glossary • Key Terms & Terminology

This glossary collects every project-specific term used across **vaish4991/notification-template-engine** — from core rendering concepts to the security fixes, delivery channels, and testing practices introduced in the recent update. Terms are grouped by category and color-coded for quick scanning.

Core Concepts

Locale	A language/region code (e.g. "en", "hi", "te") that determines which language a notification is rendered in.
Template	A text or HTML file containing static content plus placeholders, used as the source for a rendered notification.
Placeholder	A token like <code>{{name}}</code> , <code>{{date}}</code> , or <code>{{time}}</code> inside a template that gets replaced with a real value at render time.
Rendering Engine	The code path that scans a template and substitutes placeholders with actual values, producing the final message.
Locale Fallback	Automatically using the English ("en") template/locale when the requested locale isn't supported, logged as a warning rather than happening silently.
Event	The type of notification being generated, e.g. <code>interview_scheduled</code> , <code>interview_reminder</code> , <code>interview_cancelled</code> , <code>interview_completed</code> .
Notification Payload / Data	The dictionary of dynamic values (e.g. date, time) supplied to fill in a template's placeholders.

Architecture & Delivery

Notification Channel	An abstraction (<code>NotificationChannel</code>) representing a way to deliver a rendered message: console, email, or SMS.
Console Channel	A delivery channel that prints the rendered notification to standard output, mainly for local demos and debugging.

Email Channel	A delivery channel that sends the rendered HTML notification via SMTP using Python's <code>smtplib</code> .
SMS Channel	A delivery channel that sends the rendered text notification through a pluggable SMS gateway function.
Dispatch	The act of actually delivering a rendered notification to the user through a chosen channel, as opposed to just generating text.
Dry Run	A safe mode (<code>dry_run=True</code>) where a channel logs what it <i>would</i> send instead of making a real network call, so the project runs without credentials.
Channel Registry	A lookup table (<code>CHANNEL_REGISTRY</code>) mapping channel names to their implementing classes.

Security & Validation

Path Traversal	An attack where crafted input (like <code>"../etc/passwd"</code>) is used to escape an intended directory; blocked here by strict identifier validation.
Safe Identifier Validation	Restricting locale/event values to a strict pattern (letters, numbers, underscores only) before they're used to build a filesystem path.
Templates Root	The absolute, resolved base directory (<code>TEMPLATES_ROOT</code>) that every loaded template path is verified to stay inside.
HTML Escaping	Converting special characters (like <code><</code> and <code>></code>) in user-supplied values into safe HTML entities before inserting them into an HTML template, preventing markup/script injection.
Strict Locale Mode	An opt-in User setting (<code>strict_locale=True</code>) that raises an error for unsupported locales instead of silently falling back.
Input Validation	Checks applied to user data (name, email, locale) and notification data (date, time) before they are used, to prevent invalid or malicious values from propagating.

Errors & Exceptions

NotificationError	The base exception class for all notification-engine-specific errors.
MissingDataError	Raised when a template references a placeholder that has no corresponding value, replacing an earlier unhandled <code>KeyError</code> .

TemplateRenderError	A general error type for template rendering failures beyond missing data.
InvalidTemplateError	Raised when a loaded template fails validation, e.g. it's empty or contains unsupported placeholders.
DeliveryError	Raised by a channel when actual dispatch (e.g. SMTP send) fails.
FileNotFoundError	Raised when no template file exists for a given (validated) locale/event/channel combination, even after falling back to English.

Testing & Quality

Unit Test	An automated, isolated test verifying one specific behavior, written using Python's unittest framework.
Regression Test	A test added specifically to make sure a previously fixed bug (e.g. a raw KeyError) never resurfaces.
Test Coverage	The breadth of behaviors exercised by the test suite; expanded here to 26 cases across rendering, security, and delivery.
Assertion	A check within a test (e.g. assertIn, assertRaises) that fails the test if the expected condition isn't met.
Dry-Run Testing	Testing email/SMS dispatch logic without a real network connection, by relying on each channel's dry_run mode.

Formats & Tech

SMTP	Simple Mail Transfer Protocol — the protocol used by EmailChannel (via smtplib) to send real emails.
Single-Pass Substitution	Resolving all placeholders in one scan of the template using re.sub() with a lookup callback, instead of one str.replace() call per placeholder.
Channel Template Type	The template variant ("text" or "email"/HTML) selected automatically based on which delivery channel is being used.
Unicode Support	The ability of templates and rendering to correctly handle non-ASCII scripts, such as Hindi (Devanagari) and Telugu.
Logging	Structured runtime messages (INFO/WARNING/ERROR) recorded via Python's logging module for template loads, fallbacks, and delivery attempts.

